

AI-Powered Personalized Sales Email Generation System

Project Submitted by

Donepudi Tvisha Sri	AP23110011381
Parepalli Ramya	AP23110011451
Naveena Kunduru	AP23110011418
Sruthi sri paladugu	AP23110011210
Harshitha chekuri	AP23110011086



Department of Computer Science and Engineering
SRM University-AP
Amaravati, Andhra Pradesh
2025-2026

AI-Powered Personalized Sales Email Generation System

Project Description

1. Project Overview

SalesMail AI is an intelligent, AI-powered platform designed to help sales teams generate highly personalized emails using customer data and behavioral insights. The system processes large-scale retail datasets (over 99,000 customers) and transforms raw data into actionable intelligence such as customer summaries, segmentation, and sales strategies.

The application integrates advanced AI models to generate customized sales emails based on customer attributes like spending patterns, purchase history, and engagement level. Additionally, it supports offline functionality through a demo mode, ensuring usability even without API access.

2. Problem Statement

Traditional sales outreach methods are often generic and inefficient, leading to low engagement and conversion rates. Sales teams face several challenges, including difficulty in personalizing communication at scale, analyzing large customer datasets effectively, identifying high-value opportunities quickly, and generating relevant and engaging emails.

This project addresses these challenges by combining data analytics and AI-driven automation to improve sales productivity and customer engagement.

3. Objectives

Primary Objectives

- Develop an AI-based system for generating personalized sales emails
- Enable intelligent customer segmentation and targeting
- Provide real-time customer insights and recommendations

Secondary Objectives

- Reduce manual effort in email creation
- Improve conversion rates through personalization
- Enable offline functionality using rule-based logic
- Optimize performance using caching and efficient data handling

4. Core Technologies Used

Technology	Purpose
Python	Backend logic and data processing
FastAPI	REST API development
Pandas	Data analysis and CSV processing
React + Vite	Frontend UI development
Tailwind CSS	Styling and responsive design
Google Gemini	AI email generation
FastMCP	AI tool protocol (optional)
In-memory Cache	Performance optimization

5. Key Features

Customer Data Management

- Handles over 99,000 customer records
- Supports filtering, searching, and segmentation
- Displays customer attributes such as spending, category, and activity

Search & Filtering

- Filter by category, spend, and segment
- Advanced query using Pandas

AI Customer Intelligence Card

- Generates detailed customer summaries
- Identifies potential sales opportunities
- Suggests suitable email type and tone
- Classifies customers into value tiers

Email Generation

- 6 email types (Upsell, Follow-up, etc.)
- 3 tone styles
- Single and bulk generation

Caching System

- Stores generated emails and summaries
- Reduces repeated API calls
- Enhances overall system performance

Demo Mode Fallback Mechanism

- Operates without AI APIs
- Uses rule-based logic
- Supports complete offline functionality

Fallback Mechanism

- Automatically switches to rule-based output if AI fails

6. Target Users Fallback Mechanism

- Sales teams and marketing professionals
- E-commerce businesses
- CRM analysts
- Startups focusing on customer engagement

7. Real-World Applications

The SalesMail AI system has wide applicability in real-world business environments, especially in industries where customer engagement and personalized communication play a crucial role. One of the primary applications is in retail and e-commerce platforms, where businesses deal with a large number of customers and transactions. The system can analyze customer purchase history and automatically generate personalized emails such as product recommendations, promotional offers, and re-engagement messages, helping businesses increase sales and customer retention.

In marketing and sales organizations, the system can be used to automate email campaigns. Instead of manually drafting emails for each customer, sales teams can generate bulk personalized emails instantly, saving time and improving productivity. The AI-generated

summaries also help sales representatives understand customer behavior, allowing them to target the right audience with appropriate strategies.

The system is also useful in Customer Relationship Management (CRM) systems, where maintaining strong relationships with customers is essential. By integrating this application with CRM platforms, companies can automatically send follow-ups, thank-you emails, and reminders based on customer activity, improving customer satisfaction and loyalty.

Overall, the SalesMail AI system can be applied across multiple domains to automate communication, improve customer engagement, and enhance business efficiency, making it a valuable tool in modern digital ecosystems.

APPLICATION SCENARIOS / USE CASES:

Scenario 1: Marketing Campaign Optimization

A marketing team uses the system to generate personalized emails for different customer segments. By analyzing customer spending patterns and preferences, the system generates targeted emails that improve engagement and conversion rates. Bulk email generation allows the team to efficiently run large-scale campaigns with minimal manual effort.

Scenario 2: Sales Team Productivity Enhancement

A sales manager uses the system to identify high-value customers and generate tailored communication strategies. The AI-generated summaries provide insights into customer behavior, helping the team prioritize leads and improve sales performance.

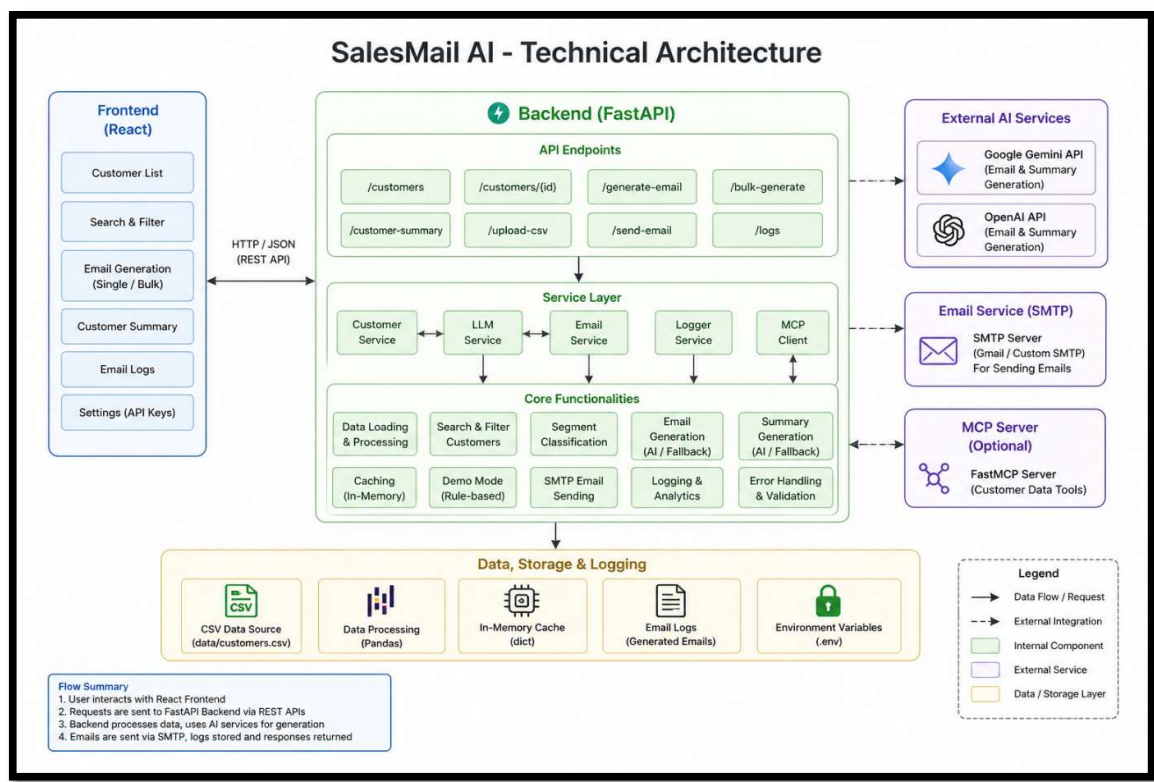
Scenario 3: Customer Retention Strategy

The system identifies inactive customers and generates re-engagement emails with personalized offers and recommendations. This helps businesses retain customers and improve long-term relationships.

Scenario 4: Automated Email Campaigns

Organizations can automate their email communication process by integrating the system with their workflow. The system continuously processes customer data and generates relevant emails, reducing dependency on manual intervention.

TECHNICAL ARCHITECTURE :



Frontend Layer

The Frontend Layer is responsible for handling all user interactions and presenting data in a clean, intuitive interface. It includes components for customer data display, dashboards, and forms where users can input or modify information. This layer also provides an email generation interface, allowing users to preview, edit, and trigger AI-generated emails before sending. Built using modern web technologies, it ensures responsiveness, usability, and a smooth user experience across devices.

Backend Layer

The Backend Layer is developed using FastAPI, which enables high-performance API development. It acts as the central processing unit of the system, handling incoming requests from the frontend, applying business logic, and coordinating between different modules. This layer manages AI services, processes datasets, and ensures secure communication between components.

Data Layer

The Data Layer is responsible for storing and managing the dataset, which is maintained in CSV format. Using Pandas, the system performs data cleaning, transformation, filtering, and aggregation. This layer ensures that structured data is readily available for analysis and AI processing. It may also include preprocessing pipelines to handle missing values, normalize fields, and prepare customer data for generating meaningful insights and personalized outputs.

AI Layer

The AI Layer leverages advanced Large Language Models such as Gemini and OpenAI GPT models to generate intelligent outputs. This includes summarizing customer data, generating personalized email content, and extracting actionable insights. The layer is designed to take structured input from the backend, craft effective prompts, and return high-quality, context-aware responses. It may also include prompt engineering techniques and response validation to ensure accuracy and relevance.

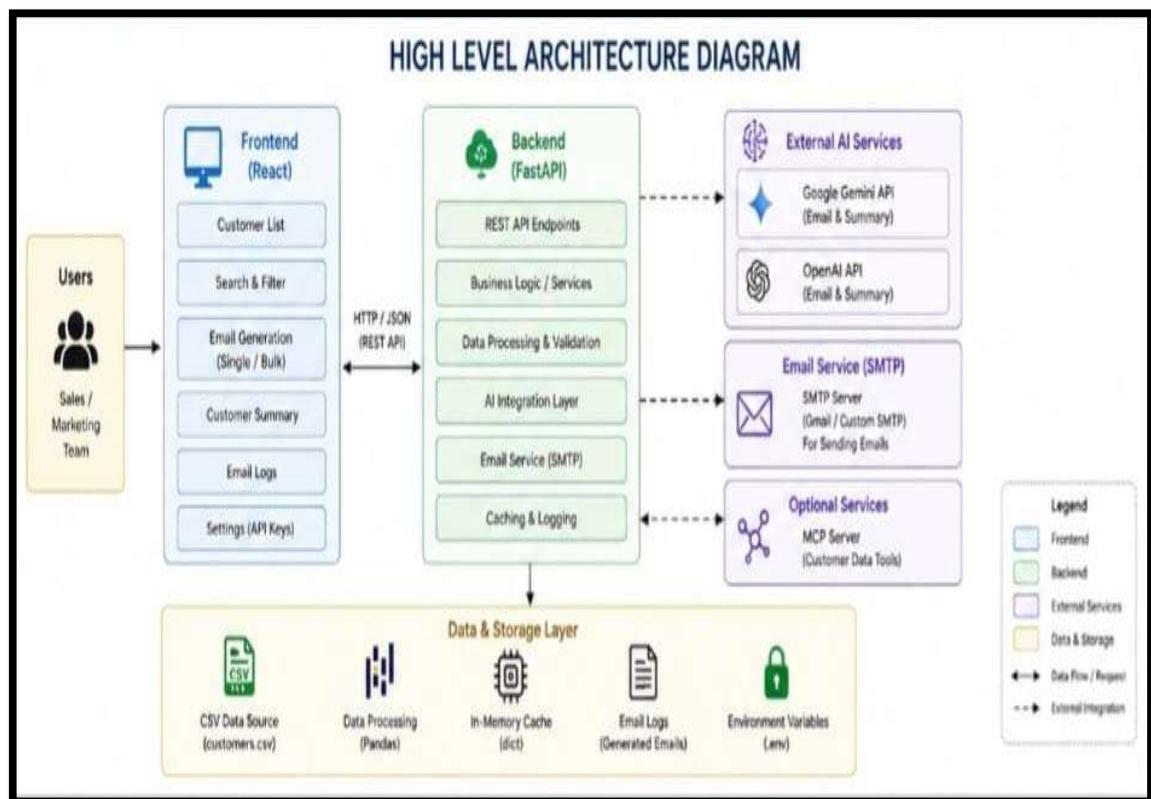
External Services

The system integrates with external services to extend its functionality. An SMTP (Simple Mail Transfer Protocol) service is used for sending emails generated by the AI layer to customers. This integration ensures reliable email delivery.

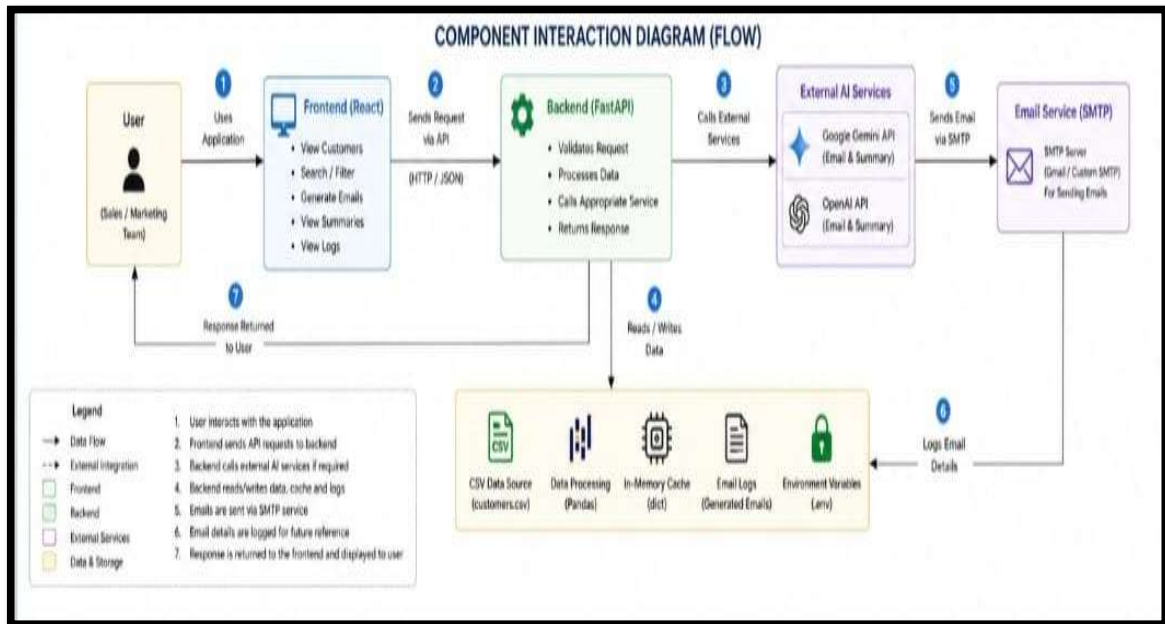
Supporting Components

- **Caching Layer:** Stores previously generated emails to reduce API cost and latency
- **Fallback Mechanism:** Rule-based generation when AI is unavailable

High-Level Architecture Diagram



Component Interaction Diagram



Pre-requisites

Software Requirements

- **Python:** Version 3.9 or higher (Python 3.10 recommended for better compatibility and performance).
- **pip:** Python package manager used to install and manage project dependencies.
- **IDE:** Visual Studio Code or Cursor IDE for development and debugging.
- **Browser:** Modern web browsers such as Chrome, Edge, or Firefox supporting ES6 features.
- **Node.js & npm:** Required for running the React frontend application.
- **Git (Optional):** Used for version control and project management.

Libraries / Frameworks / Dependencies

Install required backend dependencies using:

```
pip install fastapi uvicorn pandas python-dotenv requests python-multipart
```

Backend Libraries

- **FastAPI**
Used to build high-performance REST APIs for handling operations such as customer data retrieval, email generation, summaries, and CSV uploads. It supports asynchronous programming, making the system faster and more efficient.
- **Uvicorn**
ASGI server used to run the FastAPI application. It ensures smooth handling of API requests and supports asynchronous execution.
- **Pandas**
Used for data processing and analysis. It helps in reading CSV files, filtering customer data, performing segmentation, and handling large datasets efficiently.
- **python-dotenv**
Used to load environment variables from a `.env` file. It securely manages sensitive data such as API keys, SMTP credentials, and configuration settings.
- **Requests**
Used to make external API calls to AI services like Google Gemini or OpenAI for generating personalized emails and summaries.
- **python-multipart**
Required for handling file uploads, especially for the CSV upload feature in the system.

Frontend Dependencies

Install frontend dependencies using:

```
npm install
```

Frontend Technologies

- **React (with Vite)**
Used to build a dynamic and interactive user interface. It allows component-based architecture for better code organization and reusability.
- **Tailwind CSS**
Used for styling and creating responsive UI components. It helps in designing a clean and modern interface quickly.

Hardware Requirements

- **CPU:** Minimum Intel Core i5 or AMD Ryzen 5 processor (2 GHz or higher recommended).
- **RAM:** Minimum 8 GB RAM recommended for smooth performance while handling large datasets and AI operations.
- **Storage:** At least 1–2 GB free disk space for project files and dependencies.
- **Network:** Internet connection required for AI API calls (Gemini/OpenAI) and sending emails via SMTP.

Prior Knowledge Required

- **Programming Fundamentals**
Basic understanding of Python programming including variables, loops, functions, and modules.
- **Web Development Basics**
Knowledge of REST APIs, HTTP methods (GET, POST), and client-server architecture.
- **FastAPI Basics**
Understanding of API routing, request handling, and response structures.
- **Frontend Basics**
Basic knowledge of React and component-based UI development.
- **Data Handling**
Familiarity with CSV files and basic data processing concepts.
- **AI/LLM Concepts (Basic)**
Basic understanding of how AI models generate text and how prompts are used to guide outputs.

PROJECT OBJECTIVES

Technical Objectives

- Design and develop a scalable AI-powered sales email generation system using FastAPI as the backend framework
- Implement efficient data processing pipelines using Pandas to analyze customer transaction data and generate structured customer profiles

- Build a customer segmentation engine to classify users into categories (e.g., high-value, medium-value, low-value) based on behavioral data
- Integrate Gemini 2.5 Flash Lite for generating personalized marketing emails and customer insights
- Utilize MCP (Model Context Protocol) to enable dynamic interaction between the AI model and backend tools/data
- Develop modular APIs for:
 - customer data retrieval
 - segmentation
 - email generation
 - bulk processing

Performance Objectives

- Ensure fast API response times for real-time email generation and customer data retrieval
- Maintain efficient data processing for customer datasets using optimized Pandas operations
- Achieve high-quality and context-aware AI-generated emails through structured prompt design
- Optimize system for low latency AI responses using Gemini Flash Lite
- Reduce redundant computations using caching mechanisms for repeated requests

Deployment Objectives

- Enable cloud-based deployment for scalable AI-powered backend services
- Ensure secure handling of API keys and configurations using environment variables (.env)
- Support frontend integration (React-based UI) through RESTful APIs
- Provide easy setup and configuration for both development and production environments
- Maintain high availability and reliability of backend services

Learning Outcomes

- Gain hands-on experience in building a real-world AI-powered marketing automation system
- Understand integration of backend frameworks like FastAPI with AI models and data processing tools
- Learn how to apply LLMs for personalization and content generation
- Develop skills in API design, system architecture, and AI-driven workflows
- Improve problem-solving using data-driven insights combined with AI capabilities

Project WorkFlow:

Milestone 1: Requirement Analysis & System Design

- **Activity 1.1:** Problem Definition
- **Activity 1.2:** Functional Requirements
- **Activity 1.3:** Non-Functional Requirements
- **Activity 1.4:** System Design Decisions & Technology Stack Selection

Milestone 2: Environment Setup & Initial Configuration

- **Activity 2.1:** Development Environment Setup
- **Activity 2.2:** Dependency Installation
- **Activity 2.3:** Project Structure Creation
- **Activity 2.4:** Configuration Setup

Milestone 3: Data Setup & Processing

- **Activity 3.1:** Dataset Preparation
- **Activity 3.2:** Data Processing & Feature Engineering
- **Activity 3.3:** Data Storage & Access

Milestone 4: Core System Development

- **Activity 4.1:** Customer Management Module
- **Activity 4.2:** AI Email Generation Module
- **Activity 4.3:** Customer Summary & Strategy Module
- **Activity 4.4:** API Endpoint Development

Milestone 5: Integration & Optimization

- **Activity 5.1:** Component Integration (Frontend, Backend, Database)
- **Activity 5.2:** Performance Optimization
- **Activity 5.3:** Security Enhancements
- **Activity 5.4:** Error Handling & Validation

Milestone 6: Deployment

- **Activity 6.1:** Backend Deployment.
- **Activity 6.2:** Frontend Deployment.

Milestone 7: Testing & Validation

- **Activity 7.1:** Functional Test Case Execution
- **Activity 7.2:** Unit & Integration Testing
- **Activity 7.3:** User Acceptance Testing
- **Activity 7.4:** Performance Evaluation

Milestone 8: Deployment

- **Activity 8.1:** Deployment Architecture Setup
- **Activity 8.2:** Hosting Platform Configuration (Frontend & Backend)
- **Activity 8.3:** Deployment Implementation & Monitoring
- **Activity 8.4:** Production Deployment Considerations.

Milestone 1: Requirement Analysis & System Design

Activity 1.1: Problem Definition

Identify the need for an intelligent sales automation system that not only generates emails but also analyzes customer behavior to provide personalized communication strategies. Traditional email marketing systems rely on generic templates and manual effort, leading to low engagement and inefficiency.

The system is designed to address these limitations by integrating customer data processing, AI-driven insights, and automated email generation. It ensures scalability, accuracy, and intelligent decision-making by leveraging structured datasets and Large Language Models (LLMs).

Activity 1.2: Functional Requirements

FR-01: Allow users to view and manage a large dataset of customers loaded from CSV files.

FR-02: Enable searching and filtering of customers based on attributes such as name, category, spend, and segment.

FR-03: Generate AI-based customer summaries including value tier, behavior insights, and sales opportunities.

FR-04: Automatically recommend the best email type and tone for each customer.

FR-05: Generate personalized emails for individual customers using AI models.

FR-06: Support bulk email generation for multiple customers simultaneously.

FR-07: Provide CSV upload functionality to dynamically change datasets.

FR-08: Allow sending generated emails using SMTP configuration.

FR-09: Implement caching to reduce repeated AI calls and improve performance.

FR-10: Provide fallback rule-based output when AI services fail.

Activity 1.3: Non-Functional Requirements

NFR-01 Performance: The system should provide fast API responses and efficient email generation even for bulk operations.

NFR-02 Reliability: All endpoints must return structured responses and handle errors gracefully without crashing.

NFR-03 Scalability: The system should support large datasets (99,000+ customers) and multiple concurrent requests.

NFR-04 Security: Sensitive data such as API keys and SMTP credentials must be securely stored using environment variables.

NFR-05 Maintainability: The system should follow modular architecture for easy updates and feature additions.

NFR-06 Usability: The interface should be simple, intuitive, and user-friendly for non-technical users.

Activity 1.4: System Design Decisions & Technology Stack Selection

Backend Framework: FastAPI is used for building high-performance REST APIs with asynchronous support.

Frontend Framework: React with Vite is used for building a responsive and interactive user interface.

Data Processing: Pandas is used for efficient filtering, searching, and transformation of customer data.

AI Integration: Google Gemini and OpenAI models are used for generating summaries and emails.

Caching Mechanism: In-memory Python dictionaries are used to store generated outputs and reduce API calls.

Fallback System: Rule-based logic ensures continuous functionality even when AI services fail.

API-First Design: All functionalities are exposed via REST APIs for seamless frontend integration.

Environment Configuration: Sensitive data is stored securely using .env files.

Milestone 2: Environment Setup & Initial Configuration

Activity 2.1: Development Environment Setup

- Install Python (version 3.10 or higher) and ensure it is added to the system PATH.
- Install Node.js for frontend development.
- Set up a development environment using Visual Studio Code.
- Create the project directory and initialize the backend and frontend structure.
- Create and activate a virtual environment to isolate dependencies.

Activity 2.2: Dependency Installation

Install backend dependencies using pip:

`pip install fastapi uvicorn pandas openai python-dotenv requests python-multipart`

```
Ramya@DESKTOP-J13ETFO MINGW64 ~/Downloads/genai_mailgen (2)
$ pip install fastapi uvicorn pandas openai python-dotenv requests python-multipart
Requirement already satisfied: fastapi in C:\Users\Ramya\AppData\Local\Programs\Python\Python311\Lib\
site-packages (0.104.1)
Requirement already satisfied: uvicorn in C:\Users\Ramya\AppData\Local\Programs\Python\Python311\Lib\
site-packages (0.24.0)
Requirement already satisfied: pandas in C:\Users\Ramya\AppData\Local\Programs\Python\Python311\Lib\
site-packages (2.1.3)
Requirement already satisfied: openai in C:\Users\Ramya\AppData\Local\Programs\Python\Python311\Lib\
site-packages (2.32.0)
Requirement already satisfied: python-dotenv in C:\Users\Ramya\AppData\Local\Programs\Python\Python3
11\Lib\site-packages (1.2.1)
Requirement already satisfied: requests in C:\Users\Ramya\AppData\Local\Programs\Python\Python311\Li
b\site-packages (2.32.3)
```

Install frontend dependencies:

`npm install`

```
Ramya@DESKTOP-J13ETFO MINGW64 ~/Downloads/genai_mailgen (2)/web
$ npm install

added 3 packages, removed 6 packages, changed 16 packages, and audited 169 packages in 8s

39 packages are looking for funding
  run `npm fund` for details

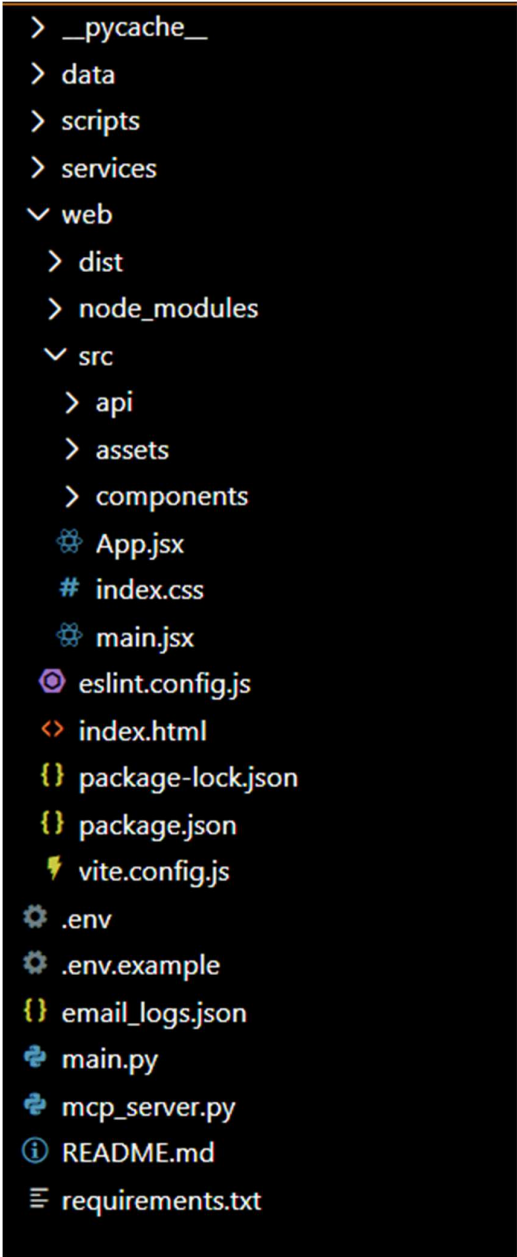
found 0 vulnerabilities
```

Activity 2.3: Project Structure Creation

The project structure of SalesMail AI is designed in a modular and organized way to ensure clarity and easy development. The application is divided into folders for backend, frontend, data, and scripts, allowing each part to be managed independently.

The backend uses FastAPI for handling APIs and business logic, while the frontend is built with React using reusable components. Customer data is stored in CSV format and processed using Pandas for efficient handling.

Configuration files like .env are used to securely manage API keys and settings. This structured approach improves maintainability and makes the system easy to extend in the future.



```
> __pycache__
> data
> scripts
> services
▼ web
  > dist
  > node_modules
▼ src
  > api
  > assets
  > components
  ⚙ App.jsx
  # index.css
  ⚙ main.jsx
  ⚙ eslint.config.js
  <> index.html
  {} package-lock.json
  {} package.json
  ⚡ vite.config.js
  ⚙ .env
  ⚙ .env.example
  {} email_logs.json
  ⚙ main.py
  ⚙ mcp_server.py
  ⓘ README.md
  ≡ requirements.txt
```


Activity 2.4: Configuration Setup

Create a .env file to store API keys and configuration settings.

Add variables such as:

- LLM provider configuration
- API keys
- Demo mode settings
- SMTP credentials

Ensure the backend dynamically reads environment variables during runtime.

Milestone 3: Data Setup & Processing

Activity 3.1: Dataset Preparation

Prepare customer datasets by merging multiple CSV files containing customer and transaction data.

Ensure data consistency and proper formatting for processing.

customer_id	gender	age	payment_method	amount	total_orders	last_interaction	first_purchase_date	avg_order_value	category	favorite_mall	categories_bought	name	email	last_purchase
C0011381	Female	20	Cash	20000	1	05-08-2022	05-08-2022	20000	Clothing	Metropol AVM	Clothing	Customer 0011381	hts252005@gmail.com	Clothing
C241288	Female	28	Credit Card	7502	1	05-08-2022	05-08-2022	7502	Clothing	Kanyon	Clothing	Customer C241288	c241288@example.com	Clothing
C111565	Male	21	Debit Card	5401.53	1	12-12-2021	12-12-2021	5401.53	Shoes	Forum Istanbul	Shoes	Customer C111565	c111565@example.com	Shoes
C266599	Male	20	Cash	300.08	1	09-11-2021	09-11-2021	300.08	Clothing	Metrocity	Clothing	Customer C266599	c266599@example.com	Clothing
C988172	Female	66	Credit Card	15004.25	1	16-05-2021	16-05-2021	15004.25	Shoes	Metropol AVM	Shoes	Customer C988172	c988172@example.com	Shoes
C189076	Female	53	Cash	242.4	1	24-10-2021	24-10-2021	242.4	Books	Kanyon	Books	Customer C189076	c189076@example.com	Books
C657758	Female	28	Credit Card	7502	1	24-05-2022	24-05-2022	7502	Clothing	Forum Istanbul	Clothing	Customer C657758	c657758@example.com	Clothing
C151197	Female	49	Cash	40.66	1	13-03-2022	13-03-2022	40.66	Cosmetics	Istinye Park	Cosmetics	Customer C151197	c151197@example.com	Cosmetics
C176086	Female	32	Credit Card	1200.32	1	13-01-2021	13-01-2021	1200.32	Clothing	Mall of Istanbul	Clothing	Customer C176086	c176086@example.com	Clothing
C159642	Male	69	Credit Card	2700.72	1	04-11-2021	04-11-2021	2700.72	Clothing	Metrocity	Clothing	Customer C159642	c159642@example.com	Clothing
C283361	Female	60	Credit Card	1200.32	1	22-08-2021	22-08-2021	1200.32	Clothing	Kanyon	Clothing	Customer C283361	c283361@example.com	Clothing
C240286	Female	36	Cash	20.92	1	25-12-2022	25-12-2022	20.92	Food & Beverage	Metrocity	Food & Beverage	Customer C240286	c240286@example.com	Food & Beverage
C191708	Female	29	Credit Card	15.15	1	28-10-2022	28-10-2022	15.15	Books	Emaar Square Mall	Books	Customer C191708	c191708@example.com	Books
C225330	Female	67	Debit Card	573.44	1	31-07-2022	31-07-2022	573.44	Toys	Metrocity	Toys	Customer C225330	c225330@example.com	Toys
C312861	Male	25	Cash	1200.32	1	17-11-2022	17-11-2022	1200.32	Clothing	Cevahir AVM	Clothing	Customer C312861	c312861@example.com	Clothing
C555402	Female	67	Credit Card	1200.32	1	03-06-2022	03-06-2022	1200.32	Clothing	Kanyon	Clothing	Customer C555402	c555402@example.com	Clothing
C362288	Male	24	Credit Card	15004.25	1	07-11-2021	07-11-2021	15004.25	Shoes	Viaport Outlet	Shoes	Customer C362288	c362288@example.com	Shoes
C300786	Male	65	Debit Card	60.6	1	16-01-2021	16-01-2021	60.6	Books	Metrocity	Books	Customer C300786	c300786@example.com	Books

Activity 3.2: Data Processing & Feature Engineering

Use Pandas to compute derived fields such as:

Total spending

Number of orders

Average order value

Last interaction date

Classify customers into segments such as High Value, Active, and Inactive.

```
def merge(customer_path: str, sales_path: str) -> pd.DataFrame:
    cust = pd.read_csv(customer_path, encoding="utf-8-sig")
    sales = pd.read_csv(sales_path)
    sales["invoice_date"] = pd.to_datetime(sales["invoice_date"], dayfirst=True, errors="coerce")
    sales["line_total"] = sales["quantity"] * sales["price"]
    agg = sales.groupby("customer_id").agg(
        total_spent      = ("line_total",      "sum"),
        total_orders     = ("invoice_no",      "nunique"),
        last_purchase_date = ("invoice_date",   "max"),
        first_purchase_date = ("invoice_date",  "min"),
        avg_order_value   = ("line_total",      "mean"),
        favorite_category = ("category",        lambda x: x.value_counts().index[0]),
        favorite_mall      = ("shopping_mall",   lambda x: x.value_counts().index[0]),
        categories_bought = ("category",        lambda x: ", ".join(sorted(x.unique()))),
    ).reset_index()
    agg["total_spent"] = agg["total_spent"].round(2)
    agg["avg_order_value"] = agg["avg_order_value"].round(2)
    agg["last_purchase_date"] = agg["last_purchase_date"].dt.strftime("%Y-%m-%d")
    agg["first_purchase_date"] = agg["first_purchase_date"].dt.strftime("%Y-%m-%d")
    df = cust.merge(agg, on="customer_id", how="left")
    df["age"] = df["age"].fillna(df["age"].median()).astype(int)
    df = df.rename(columns={"favorite_category": "category", "total_spent": "amount", "last_purchase_date": "last_intera
    df["name"] = df["customer_id"].apply(lambda x: f"Customer {x}")
    df["email"] = df["customer_id"].str.lower() + "@example.com"
    df["last_purchase"] = df["category"]
    return df
```

```
def classify(row) -> str:
    if row["_amount"] >= HIGH_VALUE_THRESHOLD:
        return "high-value"
    if row["_days_since"] <= RECENT_BUYER_DAYS_THRESHOLD:
        return "recent-buyer"
    if row["_days_since"] <= ACTIVE_DAYS_THRESHOLD:
        return "active"
    if row["_days_since"] >= INACTIVE_DAYS_THRESHOLD:
        return "inactive"
    return "active"
```

Activity 3.3: Data Storage & Access

Store the processed dataset in a CSV file.

Implement efficient data retrieval functions to fetch and filter customer records.

```
def search_customers(
    query: Optional[str] = None,
    category: Optional[str] = None,
    segment: Optional[str] = None,
    min_amount: Optional[float] = None,
    max_amount: Optional[float] = None,
    last_interaction_after: Optional[str] = None,
    last_interaction_before: Optional[str] = None,
) -> List[Dict[str, Any]]:
    """All active filters applied simultaneously with AND logic."""
    df = _get_segments_df(_read_dataframe())
    if df.empty:
        return []

    if query:
        q = str(query).strip().lower()
        mask = (
            df["name"].astype(str).str.lower().str.contains(q, na=False)
            | df["email"].astype(str).str.lower().str.contains(q, na=False)
            | df["customer_id"].astype(str).str.lower().str.contains(q, na=False)
        )
        df = df[mask]

    if category and "category" in df.columns:
        df = df[df["category"].astype(str).str.lower() == category.lower()]

    if segment:
        df = df[df["segment"] == segment]

    if "amount" in df.columns:
        nums = pd.to_numeric(df["amount"], errors="coerce").fillna(0)
        if min_amount is not None:
            df = df[nums >= min_amount]
        if max_amount is not None:
            df = df[nums <= max_amount]
```

Milestone 4: Core System Development

Activity 4.1: Customer Management Module

Develop APIs to list, search, and filter customers.

Implement CSV upload functionality to dynamically replace datasets.

```
export async function fetchCustomers() {
  const res = await fetch(`${API_BASE}/customers/`)
  const data = await handleResponse(res)
  return data.customers || []
}

export async function fetchCustomerSummary(customerId) {
  const res = await fetch(`${API_BASE}/customers/${encodeURIComponent(customerId)}/summary`)
  return handleResponse(res)
}

export async function searchCustomers({ q, category, segment, minAmount, maxAmount, interactionAfter, interactionBefore }) {
  const p = new URLSearchParams()
  if (q) p.set('q', q)
  if (category) p.set('category', category)
  if (segment) p.set('segment', segment)
  if (minAmount != null) p.set('min_amount', minAmount)
  if (maxAmount != null) p.set('max_amount', maxAmount)
  if (interactionAfter) p.set('interaction_after', interactionAfter)
  if (interactionBefore) p.set('interaction_before', interactionBefore)
  const res = await fetch(`${API_BASE}/customers/search/?${p}`)
  const data = await handleResponse(res)
  return data.customers || []
}

export async function fetchCategories() {
  const res = await fetch(`${API_BASE}/customers/categories/`)
  const data = await handleResponse(res)
  return data.categories || []
}

export async function fetchSegments() {
  const res = await fetch(`${API_BASE}/segments/`)
  const data = await handleResponse(res)
  return data.segments || {}
}
```

```
export async function fetchCustomersBySegment(segment) {
  const res = await fetch(`${API_BASE}/segments/${encodeURIComponent(segment)}/customers/`)
  const data = await handleResponse(res)
  return data.customers || []
}
```

Activity 4.2: AI Email Generation Module

Integrate LLM services to generate personalized emails.

Design prompts to include customer data and generate context-aware responses.

```
const handleGenerate = async (payload) => {
  setGeneratingEmail(true)
  setError('')
  setApiKeyError(false)
  try {
    const response = await generateEmail(payload)
    setGeneratedEmail(response.email || '')
  } catch (err) {
    setGeneratedEmail('')
    const msg = err.message || ''
    const isKeyErr = msg.toLowerCase().includes('api key') ||
      msg.includes('GOOGLE_API_KEY') || msg.includes('OPENAI_API_KEY')
    if (isKeyErr) setApiKeyError(true)
    else setError(msg)
  } finally {
    setGeneratingEmail(false)
  }
}
```

```
export async function generateEmail(payload) {
  const res = await fetch(`${API_BASE}/generate-email/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  })
  return handleResponse(res)
}

export async function generateBulkEmails(payload) {
  const res = await fetch(`${API_BASE}/generate-email/bulk/`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
  })
  return handleResponse(res)
}
```


Activity 4.3: Customer Summary & Strategy Module

Generate AI-based summaries including:

- Customer profile
- Value tier
- Sales opportunity
- Recommended email strategy

```
function CustomerSummary({ customer, onSuggest }) {
  const [summary, setSummary] = useState(null)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  useEffect(() => {
    if (!customer?.customer_id) { setSummary(null); return }
    let cancelled = false
    setLoading(true); setError(''); setSummary(null)

    fetchCustomerSummary(customer.customer_id)
      .then(data => { if (!cancelled) setSummary(data) })
      .catch(err => { if (!cancelled) setError(err.message) })
      .finally(() => { if (!cancelled) setLoading(false) })

    return () => { cancelled = true }
  }, [customer?.customer_id])

  if (!customer) return null
}
```

```
export async function fetchCustomerSummary(customerId) {
  const res = await fetch(`${API_BASE}/customers/${encodeURIComponent(customerId)}/summary`)
  return handleResponse(res)
}
```

Activity 4.4: API Endpoint Development

Develop REST APIs for:

- Customer data retrieval
- Email generation (single and bulk)
- Customer summaries
- Segmentation

Ensure proper request validation and response formatting.

Milestone 5: Integration & Optimization

Activity 5.1: Component Integration (Frontend, Backend, Data)

In this phase, all components of the SalesMail AI system are integrated to ensure smooth end-to-end functionality. The frontend developed using React communicates with the FastAPI backend through API calls. User actions such as searching customers, generating emails, and viewing summaries are sent as requests to the backend.

The backend processes these requests using service modules and retrieves customer data using Pandas. It also interacts with AI models to generate emails and summaries. Additionally, SMTP integration enables sending emails directly from the system. This ensures proper communication between all layers and accurate output generation.

Activity 5.2: Performance Optimization

Performance is improved by implementing an in-memory caching system that stores generated emails and summaries. This avoids repeated AI calls and reduces response time.

Efficient data handling using Pandas allows quick filtering and searching even for large datasets. Only required customer fields are sent to AI models, reducing processing time. Bulk operations are optimized so that cached results are reused wherever possible.

Activity 5.3: Security Enhancements

Sensitive information such as API keys and SMTP credentials are stored securely using environment variables in a .env file.

Input validation is applied to all API requests to ensure correct data processing. The system also limits the amount of customer data sent to AI models to maintain data privacy. Secure email transmission is ensured using SMTP with proper authentication.

Activity 5.4: Error Handling & Validation

The system includes proper error handling to ensure reliability. All API endpoints return structured responses with clear success or error messages.

If AI services fail due to network issues or quota limits, the system automatically switches to rule-based fallback logic. Input validation using Pydantic ensures correct data formats, preventing errors and improving system stability.

Milestone 6: Deployment

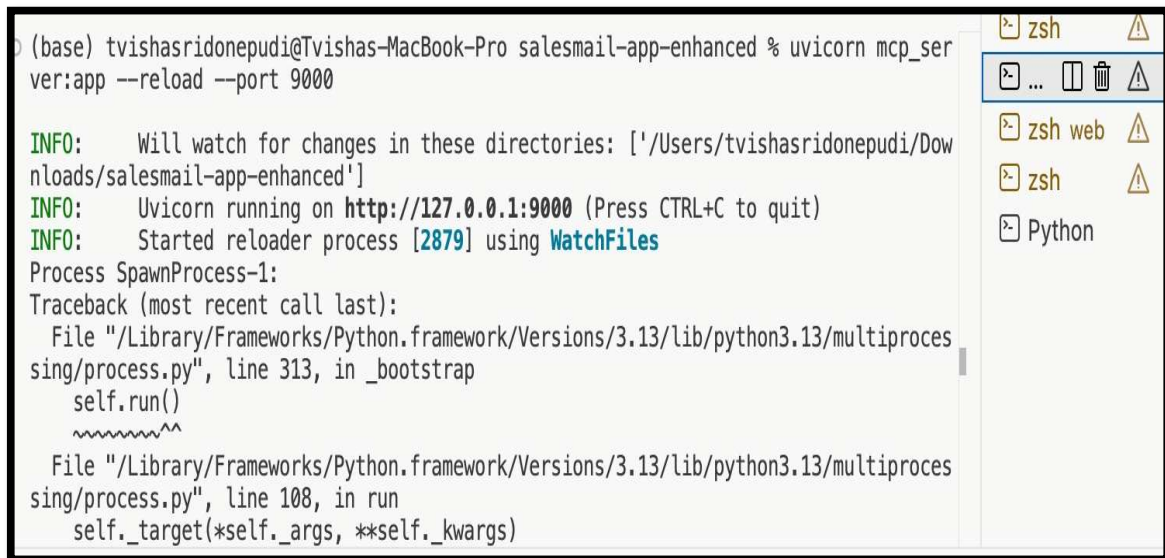
Activity 6.1: Backend Deployment

The backend is deployed using FastAPI and run with Uvicorn. It provides API endpoints for customer data, email generation, and summaries.

The backend reads configuration values from the .env file, ensuring secure handling of API keys and SMTP settings. It runs locally during development and can be deployed to cloud platforms.

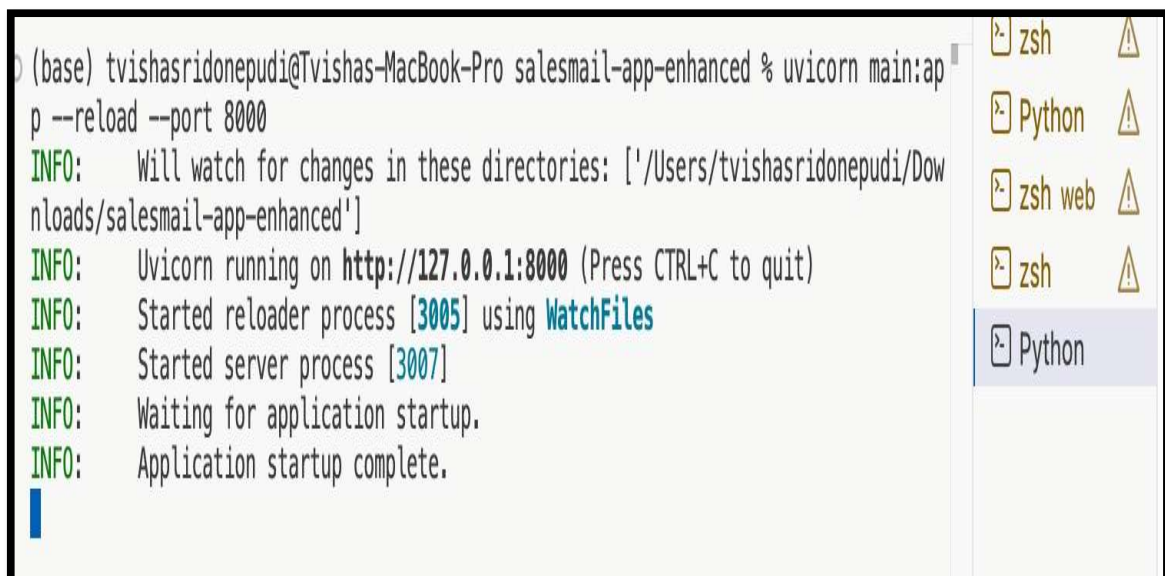
```
(base) tvishasridonepudi@Tvishas-MacBook-Pro salesmail-app-enhanced % uvicorn mcp_server:app --reload --port 9000

INFO: Will watch for changes in these directories: ['/Users/tvishasridonepudi/Downloads/salesmail-app-enhanced']
INFO: Uvicorn running on http://127.0.0.1:9000 (Press CTRL+C to quit)
INFO: Started reloader process [2879] using WatchFiles
Process SpawnProcess-1:
Traceback (most recent call last):
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/process.py", line 313, in _bootstrap
    self.run()
    ~~~~~^
  File "/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/multiprocessing/process.py", line 108, in run
    self._target(*self._args, **self._kwargs)
```

A terminal window showing the command to start Uvicorn with reload and port 9000. The output shows it's running on http://127.0.0.1:9000 and started a reloader process. A sidebar on the right lists components: zsh, Python, zsh web, and another zsh.

```
(base) tvishasridonepudi@Tvishas-MacBook-Pro salesmail-app-enhanced % uvicorn main:app --reload --port 8000

INFO: Will watch for changes in these directories: ['/Users/tvishasridonepudi/Downloads/salesmail-app-enhanced']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [3005] using WatchFiles
INFO: Started server process [3007]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

A terminal window showing the command to start Uvicorn with reload and port 8000. The output shows it's running on http://127.0.0.1:8000 and started a reloader and server process. A sidebar on the right lists components: zsh, Python, zsh web, and another zsh.

Activity 6.2: Frontend Deployment

The frontend is developed using React and runs using Vite. It provides an interactive interface for users to view customers, generate emails, and apply filters.

The frontend communicates with backend APIs to fetch data and display results. It can be deployed on platforms like Vercel for production use.

```
(base) tvishasridonepudi@Tvishas-MacBook-Pro web % npm run dev

> web@0.0.0 dev
> vite

VITE v8.0.10 ready in 312 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
11:16:22 PM [vite] (client) hmr update /src/App.jsx, /src/index.css
^C
```

Milestone 7: Testing & Validation

Activity 7.1: Functional Test Case Execution

This activity ensures that all core functionalities of the SalesMail AI system work correctly according to the defined requirements. Each feature of the application is tested individually using different input scenarios.

Test cases are executed for operations such as loading customer data, searching and filtering customers, generating single and bulk emails, creating customer summaries, uploading CSV files, and sending emails using SMTP. The outputs are verified to ensure they match expected results.

This step confirms that the system behaves correctly under normal conditions and delivers accurate and reliable outputs.

Activity 7.2: Unit & Integration Testing

In this phase, unit testing is performed on individual components such as backend functions, API endpoints, and service modules. Each function is tested independently to verify its correctness.

Integration testing is then carried out to ensure smooth communication between different components of the system. This includes interaction between frontend and backend APIs, backend and data processing modules, and backend and AI services.

This ensures that all parts of the system work together seamlessly and data flows correctly across all layers without errors.

Activity 7.3: User Acceptance Testing

User Acceptance Testing (UAT) is performed to evaluate the system from the end-user perspective. The application is tested for usability, clarity, and overall user experience.

Users interact with features such as selecting customers, generating emails, viewing summaries, and applying filters. Feedback is collected to identify improvements and ensure that the system meets user expectations.

This step ensures that the application is user-friendly and provides meaningful and understandable outputs.

Activity 7.4: Performance Evaluation

This activity focuses on evaluating the efficiency and responsiveness of the system under different conditions.

Key performance factors such as API response time, email generation speed, and system behavior during bulk operations are analyzed. The effectiveness of the caching mechanism is also evaluated to ensure reduced latency and improved performance.

The system demonstrates stable performance with fast response times and efficient handling of large datasets, ensuring a smooth user experience.

Milestone 8: Deployment

Activity 8.1: Deployment Architecture Setup

This activity involves designing the overall deployment architecture of the SalesMail AI system. The system follows a modular structure where the frontend, backend, data processing, and AI services operate as separate components connected through APIs.

The backend is deployed as a REST API service, while the frontend acts as a client interface that interacts with these APIs. This architecture ensures scalability, maintainability, and flexibility for future enhancements.

Activity 8.2: Hosting Platform Configuration (Frontend & Backend)

In this step, suitable hosting platforms are configured for deploying both frontend and backend components.

The frontend application can be deployed on platforms like Vercel, which supports fast and efficient hosting of React applications. The backend built with FastAPI can be deployed on cloud platforms such as Render or other hosting services.

Environment variables such as API keys, model configurations, and SMTP credentials are securely configured in the deployment environment to ensure proper functioning of the system.

Activity 8.3: Deployment Implementation & Monitoring

This activity involves deploying the application to the selected hosting platforms and verifying its functionality in a live environment.

Once deployed, the system is tested to ensure all features such as customer management, email generation, and summary generation work correctly. Monitoring mechanisms such as logs and analytics are used to track system performance and detect any issues.

This helps maintain system stability and ensures reliable operation after deployment.

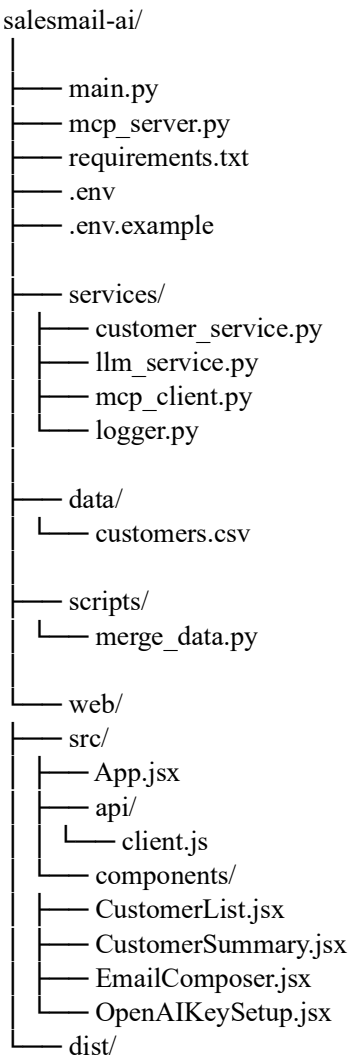
Activity 8.4: Production Deployment Considerations

This step ensures that the system is ready for real-world usage by addressing key production factors such as security, scalability, and reliability.

Sensitive data is securely managed using environment variables, and the system is designed to handle large datasets and multiple user requests efficiently. Fallback mechanisms ensure continuous operation even if AI services fail.

Proper logging and monitoring are implemented to track system activity and performance, ensuring long-term stability and maintainability.

PROJECT STRUCTURE



Folder Explanations

main.py

This is the core backend file of the application built using FastAPI. It defines all REST API endpoints such as customer retrieval, email generation, bulk operations, customer summary generation, CSV upload, and email sending.

It acts as the central controller that connects all services and handles incoming HTTP requests.

mcp_server.py

This file implements the FastMCP server, which provides tool-calling capabilities for fetching customer data. Although it is not used in the main flow, it is kept for experimental and advanced integrations with AI tools.

requirements.txt

This file contains all Python dependencies required to run the backend application. It ensures that the project can be easily set up in any environment by installing the required libraries.

.env

This file stores sensitive configuration data such as API keys, model settings, demo mode configuration, and SMTP credentials. It helps keep confidential information secure and separate from the source code.

.env.example

This file provides a template for setting up the .env file. It includes placeholder values and helps developers understand what configuration variables are required.

services/

This folder contains all backend business logic and service-layer implementations. It separates core logic from API routes, making the system modular and maintainable.

customer_service.py

Handles all customer-related operations such as:

- Loading customer data from CSV
- Searching and filtering customers
- Segment classification (High Value, Active, etc.)
- Retrieving individual customer details

It uses Pandas for efficient data processing.

Llm_service.py

This module manages all AI-related operations, including:

- Generating personalized emails
- Creating customer summaries
- Implementing caching mechanisms
- Handling fallback logic when AI fails
- Supporting demo mode for offline usage

It integrates with Google Gemini and OpenAI models.

mcp_client.py

This file contains logic for communicating with the MCP server. It is used for tool-calling experiments but is not part of the main email generation flow.

logger.py

This module logs all generated emails along with details such as timestamp, customer ID, email type, and tone. Logs are stored for analytics and tracking purposes.

data/

This folder contains the dataset used by the application.

customers.csv

This is the main dataset containing customer information such as:

Customer ID

Name

Category

Spend amount

Interaction details

The dataset is pre-processed and used for generating insights and emails.

scripts/

This folder contains utility scripts for data processing.

merge_data.py

This script is used to merge raw datasets (customer data and sales data) into a single processed CSV file. It computes derived fields such as total spend, average order value, and customer segments.

web/

This folder contains the frontend of the application built using React.

web/src/

This directory contains the source code for the frontend.

App.jsx

This is the root component of the frontend application. It manages the overall layout and integrates different UI components.

api/client.js

This file handles all API calls from the frontend to the backend. It centralizes request handling and simplifies communication between UI and server.

components/

This folder contains reusable UI components used in the application.

CustomerList.jsx

Displays the list of customers with features like search, filtering, and bulk selection.

CustomerSummary.jsx

Displays the AI-generated customer intelligence card, including summary, value tier, and recommendations.

EmailComposer.jsx

Handles email generation functionality, including selecting email type, tone, and generating single or bulk emails.

OpenAIKeySetup.jsx

Provides UI for entering and saving API keys for AI services.

dist/

This folder contains the production-ready build of the frontend application. It is generated after running the build command and used for deployment.

Result

Advantages

1.Personalized Communication

The system generates customized emails based on customer data such as purchase history, category, and spending behavior, improving customer engagement.

2.Reduced Manual Effort

Automation of email generation reduces the need for manual work. Bulk email generation allows multiple emails to be created quickly.

3.Intelligent Customer Insights

AI-generated summaries provide useful insights like value tier and sales opportunities, helping in better decision-making.

4.Efficient Data Processing

Using Pandas enables fast handling, filtering, and analysis of large datasets, ensuring smooth performance.

5.Improved Performance through Caching

In-memory caching avoids repeated AI calls, reduces response time, and improves system efficiency.

6.Offline Support (Demo Mode)

The system can run without internet or API keys using rule-based logic, making it suitable for testing and demonstrations.

7.Reliable Error Handling

Fallback mechanisms ensure the system continues to work even if AI services fail, avoiding crashes.

8.Secure Data Handling

API keys and SMTP credentials are stored securely using environment variables, protecting sensitive information.

9.Easy Integration

The system integrates frontend, backend, data processing, and AI services into a complete working solution.

10.User-Friendly Interface

The interface is simple and easy to use, allowing users to generate emails and view insights without technical knowledge.

SalesMail AI

AI-powered personalized email generation for sales teams.

• Live

Customers CSV

Upload CSV

Customers

Filters

Search by ID, name, or email...

Search

Reset

ID	NAME	CATEGORY	AMOUNT	SEGMENT
C0011381	Customer 0011381	Clothing	₹20,000	High Value
C100004	Customer C100004	Clothing	₹7,502	Inactive
C100005	Customer C100005	Shoes	₹2,400.68	Inactive
C100006	Customer C100006	Toys	₹322.56	Inactive
C100007	Customer C100007	Food & Beverage	₹1,200.00	Inactive

Showing 1-50 of 99458 customers

PrevPage 1 / 1990Next

Email Generator

Select a customer, choose type and tone, then generate.

SELECTED CUSTOMER

Click a customer in the list to select

EMAIL TYPE

Upsell

Follow-Up

Cold Outreach

Promotion

Re-Engagement

Thank-You

TOPE

Professional

Friendly

Persuasive

Generate Email

GENERATED EMAIL

Your personalized email will appear here...

Customer Intelligence

AI

HIGH VALUE TIER

SUMMARY

Customer 0011381 is a 20-year-old female who has made a single, significant purchase of \$20,000 in clothing. She was last active on August 5, 2022.

SALES OPPORTUNITY

This customer represents a high-value opportunity due to her substantial single purchase, indicating a strong potential for future large-ticket item sales.

Recommended Type

Re-Engagement

Suggested Tone

Persuasive

Apply AI Suggestions to Email Generator

Email Generator

1 customer selected — bulk mode active

↓ Export JSON

📧 Send Live Emails

SELECTED CUSTOMERS (1)

#C0011381

EMAIL TYPE

📈 Upsell

📧 Follow-Up

📧 Cold Outreach

📈 Promotion

💡 Re-Engagement

🙏 Thank-You

TONE

Professional

Friendly

Persuasive

Generate 1 Draft

1 DRAFT PREVIEW

● Customer 0011381 hts252005@gmail.com

Hey there, Customer 0011381!

It's been a little while since we last saw you, and we've missed having you around. Since your last purchase, our collection has blossomed with some incredible new styles and must-have pieces that we think you'll absolutely adore. We know you have an eye for quality, and we've curated some fresh looks we're sure will spark your fashion inspiration.

To welcome you back and help you refresh your wardrobe, we're thrilled to offer you a special incentive. Enjoy ****25% off your next order**** plus ****free express shipping**** – a little something to show we value your business. It's the perfect opportunity to explore our latest arrivals and rediscover your favorites.

Ready to dive back into style? Click here to explore our new arrivals and claim your exclusive discount: [Link to New Arrivals/Discount Page] We can't wait to see what you choose!

✓ Sent 1 email

SalesMail AI

AI-powered personalized email generation for sales teams.

Customers CSV

Upload CSV

Customers

Filters

Search by ID, name, or email...

Search

Reset

1 selected

Clear

ID	NAME	CATEGORY	AMOUNT	SEGMENT	
☒	C0011381	Customer 0011381	Clothing	₹20,000	High Value
☐	C100004	Customer C100004	Clothing	₹7,502	Inactive
☐	C100005	Customer C100005	Shoes	₹2,400.68	Inactive
☐	C100006	Customer C100006	Toys	₹322.56	Inactive
☐	C100007	Customer C100007	Food & Beverage	₹120.75	Inactive

Showing 1-50 of 99458 customers

Prev

Page 1 / 1990

Next

Customer Intelligence

Rule-based

HIGH VALUE TIER

SUMMARY

Customer 0011381 is a 20 year-old female customer in the Clothing category. Total spend: \$20,000.00 across 1 order(s). Currently inactive — last seen over 6 months ago.

SALES OPPORTUNITY

Re-engagement campaign with an incentive could recover this customer.

Recommended Type

Re-Engagement

Suggested Tone

Professional

Apply AI Suggestions to Email Generator

Email Generator

Select a customer, choose type and tone, then generate.

SELECTED CUSTOMER

Customer 0011381 · hts252005@gmail.com · #C0011381

EMAIL TYPE

Upsell

Follow-Up

Cold Outreach

Promotion

Re-Engagement

Thank-You

TONE

Professional

Friendly

Persuasive

Generate Email

GENERATED EMAIL

Copy

Send Live Email

Hi Customer 0011381,

We miss you! It's been a while since your last Clothing purchase.

As one of our valued Clothing customers, we want to make sure you're getting the most out of every purchase. Your loyalty means a great deal to us.

Please let me know if you'd like to schedule a brief call.

Best regards,

SalesMail AI Team

A message for you

Inbox x

H

hts252005@gmail.com

to me

Thu 30 Apr, 10:19 (22 hours ago)

☆

😊

↩

⋮

Hey there, Customer 0011381!

It's been a little while since we last saw you, and we've missed having you around. Since your last purchase, our collection has blossomed with some incredible new styles and must-have pieces that we think you'll absolutely adore. We know you have an eye for quality, and we've curated some fresh looks we're sure will spark your fashion inspiration.

To welcome you back and help you refresh your wardrobe, we're thrilled to offer you a special incentive. Enjoy **25% off your next order** plus **free express shipping** – a little something to show we value your business. It's the perfect opportunity to explore our latest arrivals and rediscover your favorites.

Ready to dive back into style? Click here to explore our new arrivals and claim your exclusive discount: [Link to New Arrivals/Discount Page] We can't wait to see what you choose!

Reply

Forward

😊

Limitations

1.Dependency on External APIs

The system relies on AI services like Gemini or OpenAI. If API limits are reached or services are unavailable, performance may be affected.

2.Limited Dataset Scope

The system works only with the provided dataset. It does not fetch real-time customer data from external sources.

3.Basic Rule-Based Fallback

In demo mode or fallback scenarios, emails and summaries are generated using simple rules, which may not be as accurate as AI-generated content.

4.Email Sending Constraints

Email sending depends on SMTP configuration. Incorrect settings or restrictions (like Gmail limits) may prevent emails from being delivered.

5.No Advanced Authentication

The system does not include user authentication or role-based access control, which may be required for production-level applications.

6.Static Segmentation Rules

Customer segmentation is based on predefined thresholds and does not dynamically adapt using machine learning models.

7.Limited UI Features

The frontend focuses on core functionality and lacks advanced features such as dashboards, analytics visualization, or detailed reports.

8.Scalability Constraints (Local Deployment)

When running locally, system performance may be limited by hardware resources and may not scale efficiently for very large workloads.

Future Enhancements

1.Real-Time Data Integration

The system can be enhanced to connect with live databases or CRM systems to fetch real-time customer data instead of relying only on static CSV files.

2.Advanced Personalization using Machine Learning

More advanced machine learning models can be integrated to improve customer segmentation and generate even more accurate and personalized email content.

3.User Authentication & Role Management

Adding login functionality with role-based access control (admin, user, etc.) will improve security and allow controlled access to system features.

4.Enhanced Dashboard & Analytics

A visual dashboard can be added to display insights such as customer trends, email performance, and sales analytics using charts and graphs.

5.Multi-Channel Communication

The system can be extended to support other communication channels such as SMS, WhatsApp, or push notifications along with email.

6.Improved UI/UX Design

Enhancing the frontend with better design, responsiveness, and user experience features will make the application more interactive and user-friendly.

7.Cloud Deployment & Scalability

Deploying the system on cloud platforms with scalable infrastructure will allow it to handle large-scale enterprise usage efficiently.

8.Integration with Marketing Tools

The system can be integrated with marketing platforms like CRM tools to automate campaigns and track customer interactions.

9.Feedback-Based Learning

Incorporating user feedback on generated emails can help improve AI performance over time by refining prompts and responses.

10.Enhanced Security Features

Future versions can include advanced security measures such as encryption, secure APIs, and monitoring to protect sensitive data.

Conclusion

The SalesMail AI project successfully demonstrates how artificial intelligence can be integrated with modern web technologies to automate and enhance sales communication. The system provides an efficient platform for managing customer data, generating personalized email content, and delivering intelligent insights that support better marketing strategies.

By combining FastAPI for backend development, React for frontend interaction, Pandas for data processing, and AI models for content generation, the application achieves a complete end-to-end solution. Features such as customer segmentation, AI-driven summaries, bulk email generation, caching, and fallback mechanisms improve both performance and reliability.

The system reduces manual effort, saves time, and increases the effectiveness of communication by delivering targeted and meaningful emails. It also ensures flexibility through demo mode and secure handling of sensitive data using environment variables.

Overall, the project highlights the practical application of AI in real-world business scenarios and provides a strong foundation for further enhancements such as real-time data integration, advanced analytics, and multi-channel communication.

APPENDIX

Source Code

The complete source code of the project is organized into frontend and backend modules, covering all functionalities such as customer data management, email generation, AI summaries, and API development. The code is modular, well-structured, and follows a clean architecture, making it easy to understand, maintain, and extend.

Configuration Files

Configuration files include `.env` for storing sensitive data such as API keys (Gemini/OpenAI) and SMTP credentials. Additional files like `requirements.txt` and `package.json` are used for managing backend and frontend dependencies. These files ensure smooth setup and deployment across different environments.

Dataset Details

The system uses a CSV dataset containing customer information such as customer ID, name, category, spending, and activity details. The dataset includes around **99,000 customer records**, which are processed using Pandas for filtering, segmentation, and generating insights.

API Documentation

The project provides RESTful API endpoints for all core functionalities such as customer data retrieval, search and filtering, AI summary generation, email generation (single and bulk), CSV upload, and email sending via SMTP. Each endpoint follows structured request and response formats, making it easy to integrate with frontend applications.

GitHub Repository Link

<https://github.com/Tvisha05/salesmail-app-enhanced>